CONTRA-PPO

Reinforcement Learning Agent that Plays Contra with Proximal Policy Optimization

Gymnasium · **PyTorch** · **Stable-Baselines3** · **nes-py**

Name	MSSV
Lại Hoàng Minh Phúc	SE184914
Nguyễn Mạnh Hưng	SE184693

github.com/pikamanh

THE PROBLEM

Contra: a side-scrolling shooter with a large action space

Terrain navigation

Climb, jump, and dodge obstacles while the screen auto-scrolls in real time.

Dodging attacks

Bullets and enemies spawn continuously, with no fixed pattern across playthroughs.

Defeating enemies & bosses

Aiming accurately while still keeping up the pace of movement.

Reaching the end of the stage

A long-horizon goal that requires hundreds of consecutive correct decisions.

Why isn’t rule-based AI enough?

Enemy/scroll state changes continuously and randomly based on the game's RAM — a fixed set of if/else rules cannot cover the resulting space of situations. Contra-PPO uses Deep RL: the agent learns optimal behavior (movement, shooting, dodging) directly from interaction with the environment, instead of being hand-coded with rules.

ALGORITHM

PPO Overview (Proximal Policy Optimization)

Policy Gradient

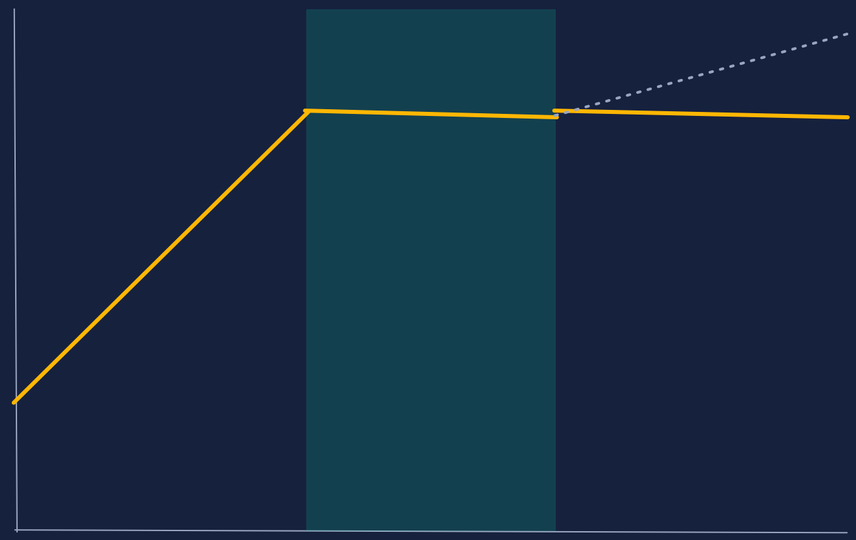
The agent has a policy network $\pi\theta(a|s)$ that outputs action probabilities directly from the observation. It is trained by increasing the probability of actions that lead to higher-than-expected reward (positive advantage), and decreasing it for negative advantage.

The problem with plain policy gradient: an update step that is too large can break an otherwise good policy, with no way to recover (no "undo").

Clipped Surrogate Objective

PPO clips the ratio between the new and old action probabilities to a small range [1-ε, 1+ε]. If the new policy tries to change too much from the old one, the reward for that change gets cut off — the update is "held back" close to the old policy, keeping training stable over thousands of updates.

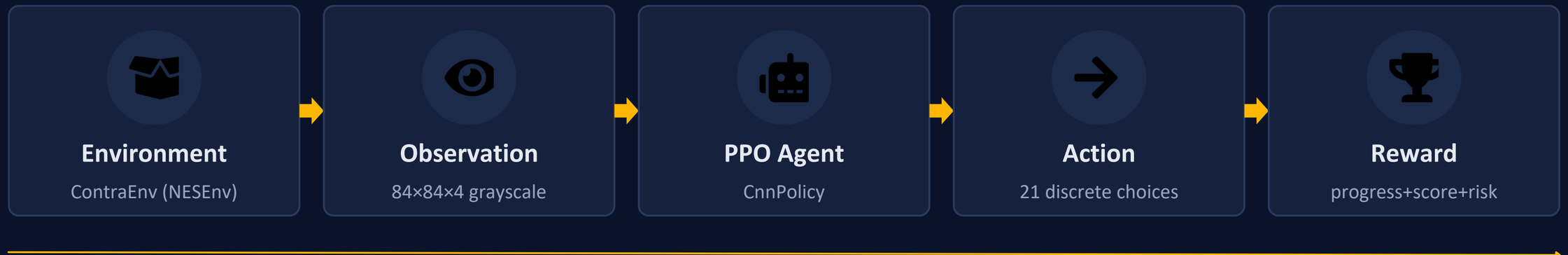
Visualizing the "safe zone" of an update step



the objective is "clipped flat" outside the safe zone — preventing the update step from going too far

ARCHITECTURE

Gymnasium env + Stable-Baselines3 pipeline



ContraEnv (NESEnv subclass, reads RAM)

JoypadSpace — 21 discrete actions (COMPLEX_MOVEMENT)

ContraGymnasiumEnv — frame-skip=4, grayscale, resize 84×84

SubprocVecEnv × N parallel → VecFrameStack (n_stack=4) → VecMonitor → VecTransposeImage

PPO (Stable-Baselines3), policy = CnnPolicy

Reward function: progress · score · dodge · boss · level

Component	Triggered when	Raw value	Actual contribution (÷10)
Progress	Moving while alive, in the normal state	clip(Δx-0.5,-4,3) + 1.5·clip(new_progress,0,4)	raw value / 10
Score	Score increases near the progress frontier (within 32px) or creates new progress	0 or +1	0 or +0.1
Dodge / survival	Alive, in the normal state, with at least 1 active enemy	+0.1	+0.01
Boss defeated	Boss-defeated flag switches from false → true	+120 (one-time)	+12 (one-time)
Stage over	End-of-stage sequence begins	+80 (one-time)	+8 (one-time)
Level advance	Current level > previous level	+50	+5

reward = (progress + score + dodge + boss + terminal + risk) / 10 — returned to the Agent every step

Risk & penalty: keeping unwanted behavior in check

Backward / wasted movement

The progress formula goes negative when moving backward or not meeting the per-step threshold.

raw: **down to -4** → **down to -0.4**

Death / life lost

Life count drops compared to the previous step — no separate penalty for taking non-lethal damage.

raw: **-15** → **-1.5**

Stagnation

Standing still > 90 steps outside the boss phase; the penalty grows progressively, disabled during the boss phase.

raw: **~ -0.017 → -2** → **~ -0.0017 → -0.2**

Game over

The game-over RAM flag is set — final failure penalty.

raw: **-35** → **-3.5**

An episode ends on game over OR when the Stage 1 boss-completion state is reached (win).

TRAINING CONFIGURATION

PPO Hyperparameters

Algorithm	**PPO**
Policy	**CNN / MLP**
Learning Rate	**3e-4**
Gamma (discount)	**0.99**
GAE Lambda	**0.95**
Clip Range	**0.2**
Batch Size	**64**
n_steps	**2048**
Total Timesteps	**Configurable**

Technical notes

train.py currently runs with default parameters that differ from the published table above: learning_rate = 5e-5, gamma = 0.9, GAE lambda (tau) = 1.0, n_steps = 512, batch_size = 128, n_epochs = 10, ent_coef = 0.02, vf_coef = 0.5, max_grad_norm = 0.5, default number of parallel envs = 32.

When resuming from a checkpoint, the default learning rate drops to 5e-5 (overridable via --resume-lr) for more stable continued training.

MAX_EPISODE_STEPS = 4500 (≈ 5 minutes of game time, with frame-skip=4 at 60fps).

TRAINING OPERATIONS

Checkpoint · Robust-best evaluation · Resume

Periodic checkpoint

CheckpointCallback saves every ~100K total timesteps to checkpoints/contra_ppo_<n>_steps.zip.

Robust-best evaluation

EvalBestCallback periodically runs 5 deterministic evaluation episodes; ranks by (min_reward, mean_reward) — favoring a stable policy over one that gets lucky in one episode then dies early in another.

Resume training

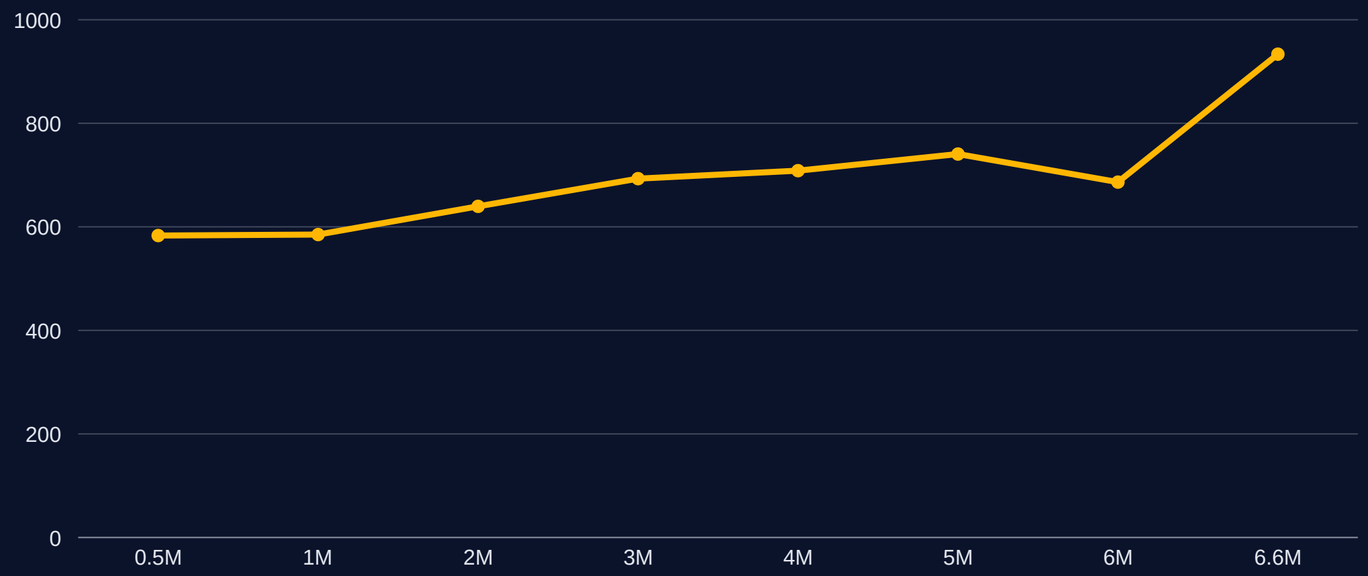
python train.py --resume <checkpoint>.zip --steps N continues the timestep count from the checkpoint; the resume learning rate defaults to 5e-5, overridable with --resume-lr.

TensorBoard logging

VecMonitor + the SB3 logger write rollout/ep_rew_mean, ep_len_mean, train/* (approx_kl, clip_fraction, explained_variance...) to logs/ for each run.

RESULTS

Training progress — log PPO_contra_1 (6.62M timesteps)



1.56K

logged episodes (rollout)

6.62M

training timesteps

+933

ep_rew_mean at end of log

Frame taken from video/.mp4 (synchronized evaluation, see training_progress_comparison.gif)*

CONCLUSION

Results achieved & directions for improvement

Achieved

- Complete Gymnasium + SB3 pipeline: env, wrapper, train, eval, resume.
- A multi-component reward function that effectively curbs backing up/standing still/farming score.
- Robust-best evaluation picks a stable checkpoint instead of a lucky one.
- The agent clearly improves over timesteps: ep_rew_mean rises from ~587 (0.5M) to ~933 (6.6M).

Directions for improvement

- Reward tuning: rebalance the progress/score/stagnation weights to reduce ep_rew_mean fluctuation around the 5–6M mark.
- Longer training / more seeds to assess stability beyond Stage 1.
- Extend to later stages instead of just the Stage 1 boss.
- Try other policy architectures (larger frame-stack, recurrent policy) for situations that need longer memory.

References

Schulman et al. (2017)

Proximal Policy Optimization Algorithms — arXiv:1707.06347

Stable-Baselines3

PyTorch implementation of PPO and other RL algorithms — Documentation

Gymnasium (Farama Foundation)

Standard RL environment API — successor to OpenAI Gym

nes-py

NES emulator for Python, the foundation for ContraEnv